NUST CHIP DESIGN CENTRE

RISC V

Project
Assembler

Name	*Waleed Amir*
Instructor	*Muhammad Imran*
Date	*26th August 2025*

1. In-Lab Tasks: (*Write your lab task & screenshots here*)

The assembler implemented was a 2 pass assembler that translates RV32I assembly code. It supports the following instructions:

1. R-type: add, sub, sll, slt, sltu, xor, srl, sra, or, and
2. I-type: addi, slti, sltiu, xori, ori, andi, lb, lh, lw, lbu, lhu, jalr, slli, srli, srai
3. S-type: sb, sh, sw
4. B-type: beq, bne, blt, bge, bltu, bgeu
5. U-type: lui, auipc
6. J-type: jal

It accepts register names either x0..x31. Immediates can be decimal or hex while branch and jump targets may be labels or numeric immediates. Output can be specified to be hex or binary by using the following flags: -h (hex) or -b (binary strings).

The assembler using a two pass process which is described below:

Two-Pass Assembly Process

Pass 1: Symbol Table Construction

- Parses the entire source file to identify labels and their addresses
- Handles .text and .data section directives
- Builds a symbol table mapping labels to memory addresses
- Processes data declarations and allocates memory space

Pass 2: Code Generation

- Translates each instruction to machine code using the symbol table
- Resolves forward references to labels
- Generates final output in the specified format

Usage:

Enter the following into the CLI in the directory where the **assembler.cpp** file is located:

g++ -std=c++17 assembler.cpp -O2 -o assembler

./asm_rv32i input.s out.hex -h

This would store the translated values of the assembly program in input.s into out.hex. This process is shown practically below:

Implementation:

prog.s(assembly to be translated):

```
.text
main:
    addi a0, zero, 5
    addi a1, zero, 3
    add a2, a0, a1
    jal ra, finish
finish:
    jal x0, finish  # infinite loop
```

Result:

```
waleed@waleed-Inspiron-5567:~/Desktop/ASsembler project$ g++ -std=c++17 assembler.cpp -O2 -o assembler
waleed@waleed-Inspiron-5567:~/Desktop/ASsembler project$ ./assembler prog.s out.hex -h
Assembled 5 instructions. Text bytes=20, data bytes=0
waleed@waleed-Inspiron-5567:~/Desktop/ASsembler project$ cat out.hex
0x00500513
0x00300593
0x00b50633
0x004000ef
0x0000006f
```

Conclusion:

This assembler project taught me how assembly code actually gets converted to machine code. I learned about two-pass system where you first scan for labels then generate the actual instructions.

The most difficult part was understanding how different instruction types work - R-type for basic operations, I-type for immediates and loads, S-type for stores, etc. Each one has its own bit pattern which was confusing.

Getting labels to work properly was tricky because you need to reference things before you've seen them in the code. That's why the two-pass approach is needed.

I also learned about bit manipulation and how registers are encoded in the instructions.

Overall this gave me a better understanding of what happens between writing code and it actually running on the processor.